

COMSATS University Islamabad, Attock Campus

Department of Computer Science



Course: SQE Lab

Lab Project

Submitted To	Madam Munazza
Submitted By	
Qazi Anwaar Ahmed Jilani	Fa24-bse-040
Syed Nazr-e-Abbas	Fa24-bse-053
Asad Ali	Fa24-bse-047

Date of Submission: May 18, 2026

Table of Contents

1. Introduction	3
2. Project Overview	3
3. Objectives	3
4. System Features	3
User Authentication	3
Quiz System	3
Leaderboard	3
5. Technologies Used	3
6. System Architecture (Flow Diagram)	4
7. Functional Requirements	4
8. Non-Functional Requirements	4
9. Modules Description	4
9.1 Authentication Module (auth.py)	4
9.2 Quiz Module (quiz.py)	5
9.3 Leaderboard Module (leaderboard.py)	7
9.4 Database Module (database.py)	8
10. Source Code Structure	8
11. Database/File Structure	9
users.json (dynamically updated)	9
leaderboard.json	9
12. Manual Test Cases	9
13. Unit Testing	9
BUG REPORT	10
BUG-01	10
BUG-02	11
BUG-03	12
14. Code Quality Analysis	12
15. Security Analysis	13
16. Future Enhancements	13
17. Conclusion	13

1. Introduction

The Online Quiz System is a Python-based application developed to conduct quizzes digitally. The system allows users to signup, login, attempt quizzes, calculate scores, and view leaderboard rankings.

The project demonstrates:

- Python programming concepts
- JSON file handling
- Unit testing
- Bug reporting
- Code quality analysis

2. Project Overview

The system provides:

- User Signup
- User Login
- Quiz Selection
- MCQ Questions
- Score Calculation
- Leaderboard Display

All data is stored using JSON files.

3. Objectives

- Develop a Python quiz application
- Implement authentication system
- Store data using JSON
- Apply functions, loops, and conditions
- Perform testing and debugging

4. System Features

User Authentication

- Signup
- Login

Quiz System

- Multiple quizzes
- MCQ questions
- Score calculation

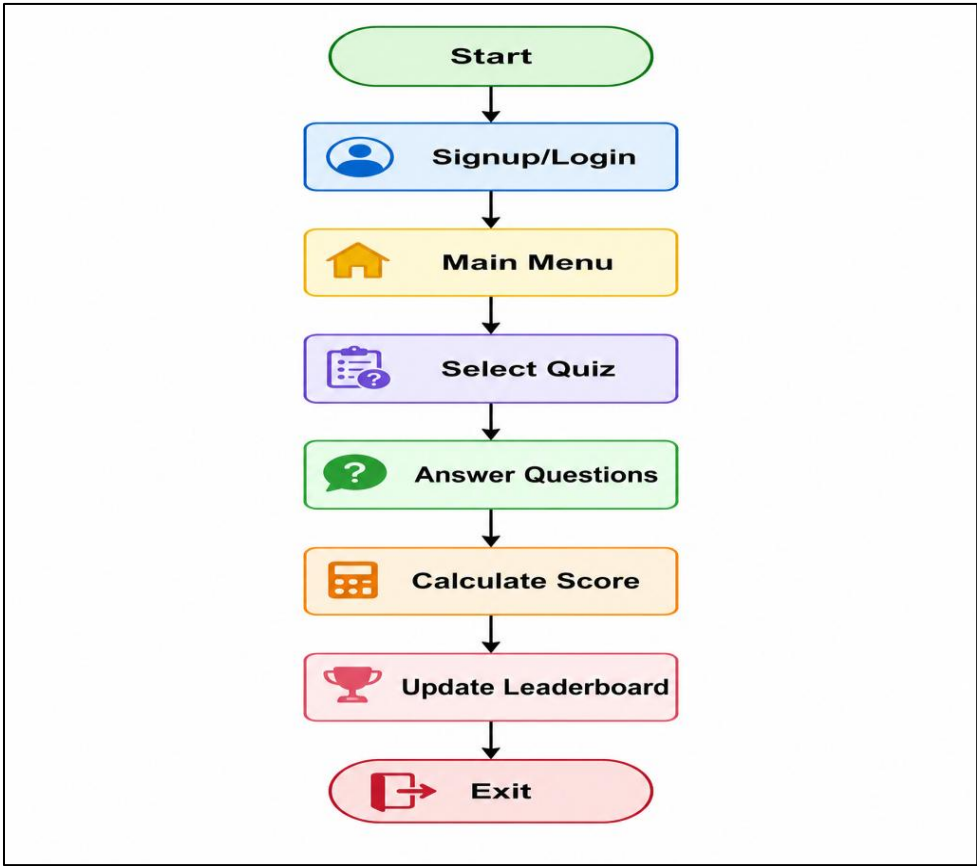
Leaderboard

- Ranking display
- Score storage

5. Technologies Used

Technology	Purpose
Python	Programming Language
JSON	Data Storage
unittest	Unit Testing
SonarQube	Code Quality Analysis

6. System Architecture (Flow Diagram)



7. Functional Requirements

ID	Requirement
FR-1	User can signup
FR-2	User can login
FR-3	User can select quiz
FR-4	System calculates score
FR-5	System updates leaderboard

8. Non-Functional Requirements

Requirement	Description
Performance	Fast execution
Reliability	Handles invalid input
Maintainability	Modular code
Usability	Easy interface

9. Modules Description

9.1 Authentication Module (auth.py)

Handles:

- Signup
- Login
- User validation

Code

```

from database import load_users, save_users

class Auth:

    def signup(self):
        users = load_users()

        username = input("Create username: ")
        password = input("Create password: ")

        for u in users:
            if u["username"] == username:
                print("User already exists!")
                return None

        users.append({"username": username, "password": password})
        save_users(users)

        print("Signup successful!")
        return username

    def login(self):
        users = load_users()

        while True:
            username = input("Username: ")
            password = input("Password: ")

            for u in users:
                if u["username"] == username and u["password"] == password:
                    print("Login successful!")
                    return username

            print("Invalid credentials, try again.\n")

```

Screenshot

Signup:

```

===== ONLINE QUIZ SYSTEM =====
1. Login
2. Signup
Choose: 2
Create username: Asad
Create password: asad12x@
Signup successful!

```

Login:

```

===== ONLINE QUIZ SYSTEM =====
1. Login
2. Signup
Choose: 1
Username: Asad
Password: asad12x@
Login successful!

```

9.2 Quiz Module (quiz.py)

Handles:

- Questions
- Answers
- Score calculation

Code

```
import time

class Quiz:

    def run_quiz(self, questions):
        score = 0
        print("\nQuiz Started! (Each question = 1 mark)")

        for q in questions:
            print("\n", q["question"])

            for opt in q["options"]:
                print("-", opt)

            start = time.time()
            ans = input("Answer: ")
            end = time.time()

            if end - start > 10:
                print("Time up!")
                continue

            if ans.lower() == q["answer"].lower():
                print("Correct! +1")
                score += 1
            else:
                print("Wrong!")

        return score

    def quiz1(self):
        return self.run_quiz([
            {
                "question": "What is the capital of Pakistan?",
                "options": ["Karachi", "Islamabad", "Lahore", "Peshawar"],
                "answer": "Islamabad"
            },
            {
                "question": "2 + 2 = ?",
                "options": ["3", "4", "5", "6"],
                "answer": "4"
            }
        ])

    def quiz2(self):
        return self.run_quiz([
            {
                "question": "HTML is used for?",
                "options": ["Web structure", "AI", "Database", "OS"],
                "answer": "Web structure"
            },
            {
                "question": "CPU stands for?",
                "options": ["Central Processing Unit", "Control Power Unit", "Central Print Unit", "None"],
                "answer": "Central Processing Unit"
            }
        ])
```

Screenshot

Quiz 1:

```
===== MENU =====
1. Quiz 1
2. Quiz 2
3. Leaderboard
4. Exit
Choose: 1

Quiz Started! (Each question = 1 mark)

What is the capital of Pakistan?
- Karachi
- Islamabad
- Lahore
- Peshawar
Answer: ISLAMABAD
Correct! +1

2 + 2 = ?
- 3
- 4
- 5
- 6
Answer: 4
Correct! +1
Score: 2
```

Quiz 2:

```
===== MENU =====
1. Quiz 1
2. Quiz 2
3. Leaderboard
4. Exit
Choose: 2

Quiz Started! (Each question = 1 mark)

HTML is used for?
- Web structure
- AI
- Database
- OS
Answer: web structure
Correct! +1

CPU stands for?
- Central Processing Unit
- Control Power Unit
- Central Print Unit
- None
Answer: central processing unit
Correct! +1
Score: 2
```

9.3 Leaderboard Module (leaderboard.py)

Handles:

- Score saving
- Ranking display

Code

```
from database import load_leaderboard, save_leaderboard

class Leaderboard:

    def add_score(self, username, score):
        data = load_leaderboard()

        found = False

        for d in data:
            if d["username"] == username:
                d["score"] += score
                found = True
                break
```

```

if not found:
    data.append({"username": username, "score": score})

save_leaderboard(data)

def display(self):
    data = load_leaderboard()
    data = sorted(data, key=lambda x: x["score"], reverse=True)

    print("\n===== LEADERBOARD =====")
    for d in data:
        print(d["username"], "->", d["score"])

```

Screenshot

```

===== LEADERBOARD =====
Asad -> 4
abbas -> 3
anwaar -> 2

```

9.4 Database Module (database.py)

Handles:

- JSON file operations

Code

```

import json

def load_users():
    try:
        with open("users.json", "r") as file:
            data = file.read().strip()
            return json.loads(data) if data else []
    except:
        return []

def save_users(users):
    with open("users.json", "w") as file:
        json.dump(users, file, indent=4)

def load_leaderboard():
    try:
        with open("leaderboard.json", "r") as file:
            data = file.read().strip()
            return json.loads(data) if data else []
    except:
        return []

def save_leaderboard(data):
    with open("leaderboard.json", "w") as file:
        json.dump(data, file, indent=4)

```

10. Source Code Structure


```
SQE_Project/
├── main.py
├── auth.py
├── quiz.py
├── leaderboard.py
├── database.py
├──
├── users.json
├── leaderboard.json
├──
└── test-quiz.py
```

11. Database/File Structure

users.json (dynamically updated)

```
[
  {
    "username": "admin",
    "password": "1234"
  }
]
```

leaderboard.json

```
[
  {
    "username": "abbas",
    "score": 3
  },
  {
    "username": "anwaar",
    "score": 2
  },
  {
    "username": "Asad",
    "score": 4
  }
]
```

12. Manual Test Cases

Test Case	Input	Expected Result	Actual Result	Remarks/Status
Signup	New user	Account created	Account created	Pass
Login	Correct credentials	Login successful	Login successful	Pass
Quiz	Select quiz	Quiz starts	Quiz starts	Pass
Score	Correct answers	Score generated	Score generated	Pass
Leaderboard	View leaderboard	Rankings displayed	Rankings displayed	Pass

13. Unit Testing

Python unittest framework was used for testing.

Code

```
import unittest
from unittest.mock import patch
from auth import Auth
from database import save_users, load_users

class TestAuth(unittest.TestCase):

    def setUp(self):
        self.auth = Auth()

        # reset users for testing
```

```
save_users([
    {"username": "admin", "password": "1234"}
])

# ----- LOGIN TESTS -----

@patch("builtins.input", side_effect=["admin", "1234"])
def test_login_success(self, mock_input):
    result = self.auth.login()
    self.assertEqual(result, "admin")

@patch("builtins.input", side_effect=["admin", "asad", "admin", "1234"])
def test_login_retry_then_success(self, mock_input):
    result = self.auth.login()
    self.assertEqual(result, "admin")

# ----- SIGNUP TESTS -----

@patch("builtins.input", side_effect=["newuser", "pass123"])
def test_signup_success(self, mock_input):
    result = self.auth.signup()
    self.assertEqual(result, "newuser")

@patch("builtins.input", side_effect=["admin", "1234"])
def test_signup_existing_user(self, mock_input):
    result = self.auth.signup()
    self.assertIsNone(result)

if __name__ == "__main__":
    unittest.main()
```

Screenshot

```
-----
Ran 4 tests in 0.040s

OK
```

14.BUG REPORT

BUG-01

Bug Name:

Generic Exception Handling in database.py

Description:

A generic `except:` block was used in the database file which can hide actual errors and reduce code quality.

Steps to Reproduce:

- 1. Open `database.py`
- 2. Locate the `load_users()` function
- 3. Observe the exception handling block

Expected Result:

The system should catch specific exceptions such as:

```
except FileNotFoundError:
```

Actual Result:

The code used:

except:

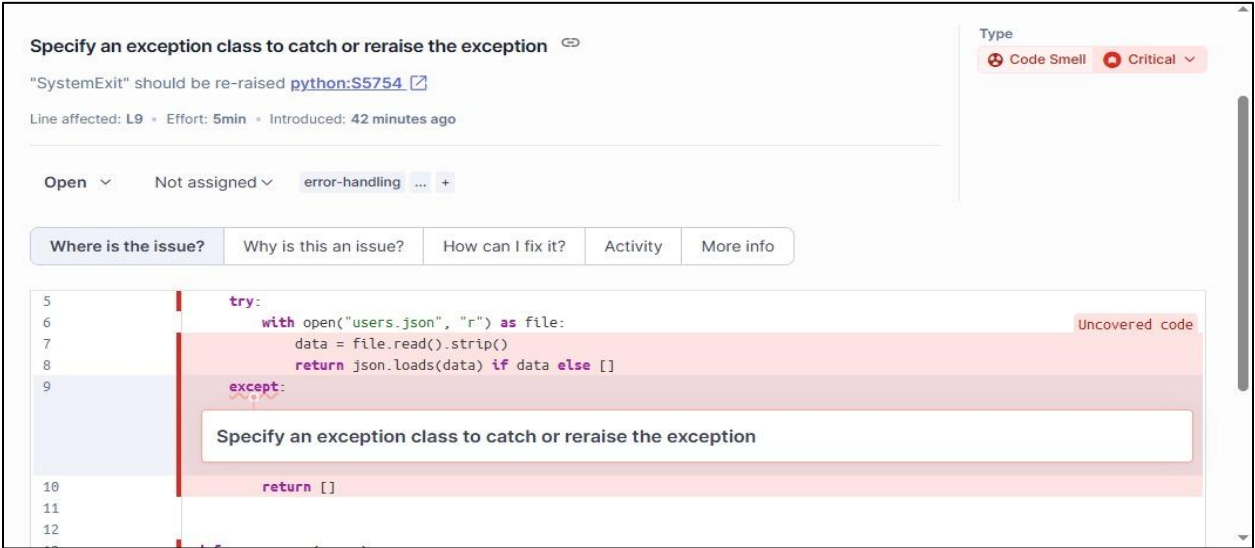
Severity:

Critical

Status (Pass/Fail):

Pass (Fixed)

Screenshot



BUG-02

Bug Name:

Duplicate Test Implementation

Description:

Two unit test functions had similar implementation which caused code duplication issue in SonarQube.

Steps to Reproduce:

- 1. Open test-quiz.py
- 2. Compare:
 - o test_login_success()
 - o test_login_retry_then_success()

Expected Result:

Each test case should have unique implementation.

Actual Result:

Both test cases contained similar logic and assertions.

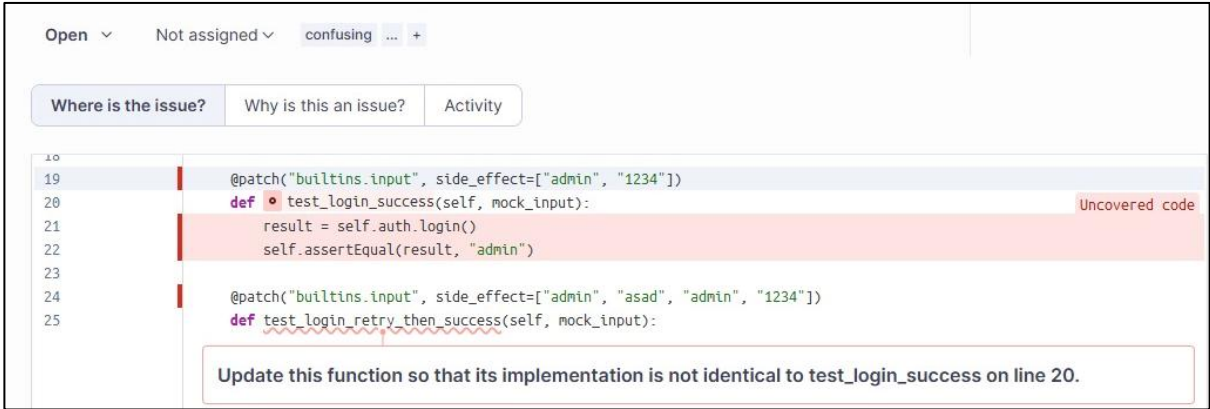
Severity:

Major

Status (Pass/Fail):

Pass (Fixed)

Screenshot



BUG-03

Bug Name:

Duplicate Username Allowed

Description:

The system previously allowed users to create accounts using existing usernames.

Steps to Reproduce:

- 1. Run the application
- 2. Signup using an already existing username
- 3. Observe system behavior

Expected Result:

System should display:

User already exists!

Actual Result:

Duplicate usernames were being added.

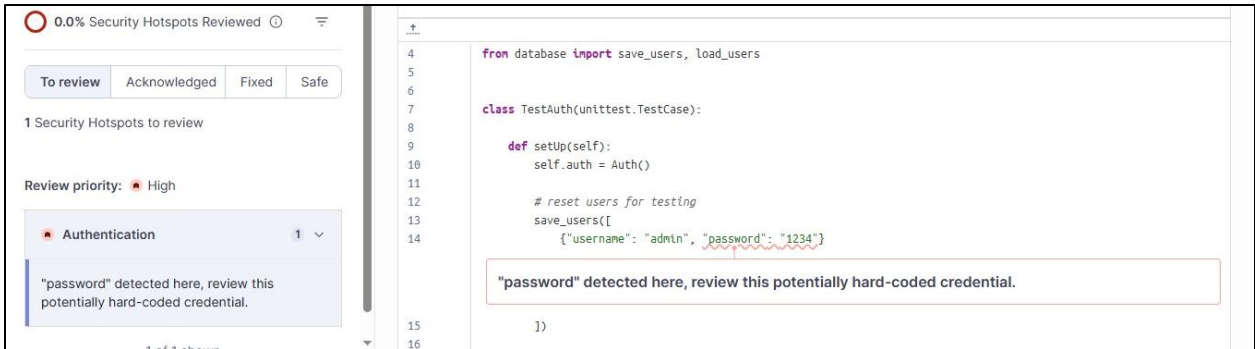
Severity:

Medium

Status (Pass/Fail):

Pass (Fixed)

Screenshot



15. Code Quality Analysis

Code quality was analyzed using SonarQube.

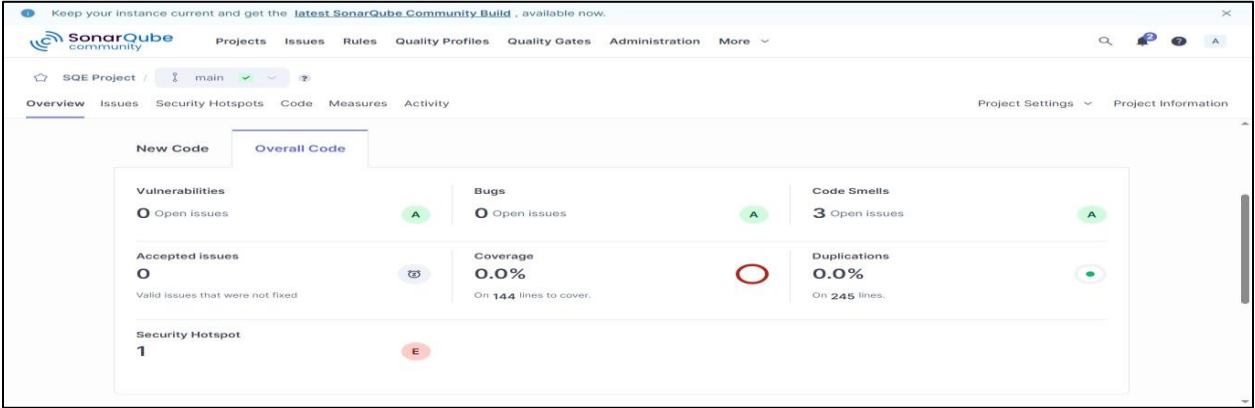
Issues found:

- Generic exception handling
- Duplicate code

Improvements:

- Added proper exception handling
- Reduced duplicate code

Screenshot



16. Security Analysis

Security issues:

- Passwords stored in plain text
- No password encryption

Suggested improvements:

- Password hashing
- Better validation

Screenshot:

17. Future Enhancements

- GUI using Tkinter
- SQLite database
- Password hashing

18. Conclusion

The Online Quiz System successfully demonstrates:

- Authentication
- Quiz management
- Score calculation
- Leaderboard system
- Testing and debugging